Documentación de Plataforma de Asistencia

Introducción

La Plataforma de Asistencia es una aplicación web que se desarrolló con 3 capas que funciona para poder registrar el pase de lista de infantes en 212 estancias del bienestar y desarrollo infantil del ISSSTE, dentro de las funciones se encuentra el seguimiento de faltas, incidencias y casos médicos que se derivan de estos registros.
Este documento tiene como función describir la arquitectura del sistema, cómo funcionan el frontend, el backend y la base de datos, la justificación de usar una base de datos no relacional como la usada (MongoDB), el consumo de memoria y CPU (aproximados) de los procesos importantes medidos desde la consola con htop.

Arquitectura

El sistema sigue una arquitectura cliente-servidor de tres capas: una aplicación de una sola página (SPA) en React Admin, una API en Node.js con Express y una base de datos MongoDB. Todo el tráfico entre capas va cifrado con HTTPS y cada petición a la API lleva el JWT del usuario.

Capa	Tecnología	Proceso en ejecución	Responsabilidad
Frontend	React 19, React Admin 5, MUI 7, TypeScript, html5-qrcode, ra-data-json-server	node vite (npm run dev) en el servidor; la aplicación corre en el navegador del usuario	Interfaz por rol, captura del pase de lista, lectura de credenciales QR, tableros
Backend	Node.js 22, Express 5, Mongoose 9, argon2, JWT, mongoose-field-encryption	node index.js	Autenticación, autorización por rol, faltas, tickets, semáforo de seguimiento, incidencias, bitácora, cifrado, cierre diario
Base de datos	MongoDB	mongod	Persistencia, índices

Flujo de una petición: pase de lista

1. La educadora inicia sesión./login verifica la contraseña con argon2id y devuelve un JWT con usuario y roles.
2. Al abrir Pase de lista, el frontend llama GET /infantes. El backend valida el JWT, busca los grupos del usuario, devuelve sus infantes y crea la asistencia del día como pendiente.
3. Al guardar, PATCH /asistencia hace un upsert por infante, fecha y turno, y registra la acción en bitacora.
4. A medianoche, cerrarDia() marca como FALTA_INJUSTIFICADA las asistencias que

siguen sin estatus.

El escaneo de credenciales sigue el mismo camino con POST /Pase_de_Lista/escaneo; el clientUuid generado en el dispositivo evita duplicados si la petición se repite.

Justificación del uso de base de datos

El sistema necesita una base de datos porque su información es compartida, persistente y auditable: 212 estancias, 7 roles y varios dispositivos por estancia leen y escriben el mismo padrón y el mismo registro de asistencia todos los días. Guardar esto en archivos o en el navegador no permitiría concurrencia, consultas agregadas ni control de acceso.

Por qué MongoDB (modelo documental)

1. Requisito explícito. La HU 33 pide optimizar la base existente con un modelo no relacional.
2. Forma de los datos de origen. El padrón de infantes trae muchos atributos opcionales y agrupados (movimiento, cambioEstrato, cambioEbdi, baja, discapacidad, diferencias). En un documento son subdocumentos; en SQL serían muchas columnas nulas o tablas 1:1 adicionales.
3. Datos que siempre se leen con su padre. Las salas de una estancia, los tutores autorizados y contactos de emergencia de un infante, los intentos de contacto de un ticket y las acciones de una incidencia se consultan juntos. Incrustarlos evita joins y deja una lectura por pantalla.
4. JSON de extremo a extremo. React Admin, Express y MongoDB usan JSON, así que no hay capa de mapeo objeto-relacional; Mongoose aporta esquemas, enumeraciones y validadores.

Costo asumido: MongoDB no impone llaves foráneas, así que la integridad referencial (por ejemplo, que un grupo.infantes[] exista en infantes) la cuida el backend. Las transacciones de varios documentos requieren réplica; el sistema actual no las usa y corre sobre una instancia única.

Modelo de datos y normalización

La base ISSTE tiene 13 colecciones definidas en Backend/models/. El modelo está normalizado en lo que crece sin límite (asistencias, tickets, notas y bitácora son colecciones propias referenciadas por ObjectId) y desnormalizado a propósito en datos acotados que se leen con su padre o que sirven como filtro frecuente. Las desviaciones se justifican abajo.

Colecciones

Colección	Entidad
estancias	EBDI (estancia)
usuarios	Personal con acceso
infantes	Niño inscrito
tutores	Padre, madre o autorizado

Colección	Entidad
grupos	Grupo de una sala por ciclo
asistencias	Registro diario por infante y turno
tickets	Seguimiento de una falta
asesorias	Orientación de Trabajo Social o Médico
notasConfidenciales	Nota médica o familiar (cifrada)
incidencias	Accidente, enfermedad o conducta en la estancia
consentimientos	Aceptación del aviso de privacidad
bitacora	Auditoría de acciones
historico	Expediente archivado de un ticket cerrado

Las referencias usan ObjectId, salvo tres llaves naturales con índice único: ebdi (número de estancia), usuario (nombre de usuario) y el nombre de sala. Las tres son estables y legibles en la bitácora.

Consumo de recursos: memoria y CPU por proceso

Con los datos de prueba, el sistema completo usa menos de 1 GB y responde en milisegundos.

Conjunto de datos	Estancias	Infantes	Asistencias	Tamaño
Demo (script BaseDatos.js del repositorio)	13	50	44	0.2 MB

Usando htop, nos dimos cuenta de que todas las peticiones usan 1% de la CPU. Esto se debe a que lo estamos probando de manera local y usando las peticiones como el GET dentro de la vista de base de datos. Aunque sean muchos datos, el proceso termina en menos de un segundo, por lo que se estaría cumpliendo la HU 33. En cuanto a memoria, por ejemplo para un administrador central tendríamos que evaluar la cantidad de: empleados, infantes y asistencias que se guardarán dentro de la base de datos para saber el tamaño de los objetos que estaría pasando la lista.